GraphAIOps: Unifying Troubleshooting Guide Generation, Multi-Agent Triage, and Graph-Based Root Cause Analysis for Cloud Incident Management

Omprakash Pugazhendhi
Department of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
omprakash.2021@vitstudent.ac.in

***Abstract*—Managing incidents in large-scale cloud systems is one of the most operationally demanding challenges in modern software engineering. On-call engineers are paged at all hours, expected to diagnose and fix outages quickly, and are often working from troubleshooting guides that are outdated, incomplete, or simply missing. This paper presents GraphAIOps, a unified synthesis of three recent Microsoft Research systems that together automate the full incident lifecycle: TSGen generates structured troubleshooting guides directly from historical incident telemetry; TRIANGLE routes each incoming alert to the correct engineering team through a multi-agent negotiation mechanism; and a Causal Knowledge Graph with graphlet inference surfaces root causes from years of post-mortem records, augmented by a fine-tuned language model. We show that graph-based reasoning is the technical thread that connects all three systems, and that treating them as a single integrated pipeline—rather than independent tools—unlocks compounding improvements that none of them achieves alone.**

***Index Terms*—cloud reliability, AIOps, incident management, knowledge graph, multi-agent LLM, troubleshooting guide generation, root cause analysis, graphlet inference**

I. Introduction

Picture an on-call engineer paged at 3 a.m. because a latency spike is affecting checkout on a production service. The first thing they reach for is the troubleshooting guide—but the guide was last updated eight months ago, references a dashboard that no longer exists, and was written for a slightly different version of the service. Meanwhile, the incident has already been routed to the wrong team by an automated classifier that misread the error signature, and re-routing it takes another twenty minutes. By the time the right engineer is looking at the right logs, the outage has lasted over an hour.

This scenario is not hypothetical. Cloud services at hyperscale generate thousands of incidents daily, and the three failure modes described above—stale documentation, mistriage, and slow root cause analysis—are the dominant reasons why mean time to resolution (MTTR) remains stubbornly high even at mature engineering organizations. Each failure mode has attracted its own line of research: AutoTSG [4] automated the execution of existing troubleshooting guides; DeCaf [6] applied machine learning to KPI-based incident routing; RCACopilot [5] introduced chain-of-thought LLM reasoning for root cause analysis. What none of these systems does is share artifacts or feed outputs from one stage into the next.

This paper synthesizes three recent Microsoft Research systems—TSGen [1], TRIANGLE [2], and KG-Graphlet RCA [3]—into a unified pipeline we call **GraphAIOps**. Each system was designed independently to solve one of the three failure modes above. We argue that when you look at them together, the architectural choices converge: all three represent knowledge as graphs, all three use graph traversal or graph attention to answer queries over that knowledge, and all three produce artifacts that the other two systems can consume. That convergence is not a coincidence—it reflects something real about the structure of incident data, which is fundamentally relational and does not fit naturally into flat retrieval or tabular classifiers.

The rest of this paper describes each system in turn (Sections III–V), then shows how they integrate into a self-reinforcing pipeline (Section VI), and closes with a discussion of what remains unsolved (Section VII).

II. Related Work

The incident management literature is large, but prior work almost uniformly addresses one stage at a time. On the documentation side, Nissist [7] converts existing human-written TSGs into knowledge graphs that can answer step-by-step queries; it improves TSG usability but does not solve the problem of TSGs that are missing or outdated to begin with. StepFly [8] ran an empirical study on 92 real TSGs and found that *instruction drift*— where an LLM agent ignores the guide structure and rewrites queries on the fly— is the primary failure mode in agentic TSG execution. LLexus [9] generates executable automation plans from human-authored TSGs, moving closer to full autonomy but still requiring humans to write the source material. FLASH addresses the

same execution problem using a ReAct-based LLM agent with dynamic context injection during runtime.

On the triage and diagnosis side, DeCaf [6] uses machine learning and pattern mining over KPI time-series data to triage incidents, without any natural language understanding of incident tickets. RCACopilot [5] goes further by applying GPT-4 with chain-of-thought prompting to generate root cause hypotheses from alert signals, but it does not exploit the relational structure of historical post-mortem data. To our knowledge, no existing system unifies automated guide generation, multi-team triage with shared knowledge, and graph-structured root cause analysis into a single integrated pipeline.

III. TSGen: Synthesizing Troubleshooting Guides from Incident History

An internal Microsoft study examined over 4,000 troubleshooting guides and found a clear pattern: when a high-quality, up-to-date TSG exists and an engineer finds it, incident mitigation time drops by roughly 40% [1]. The catch is that most TSGs do not meet that bar. They are written once, rarely maintained, stored in silos that engineers don't know to look in, and often cover the happy path without addressing the edge cases that cause the most damage in production.

TSGen attacks this problem at the source by generating TSGs from the raw material that accumulates naturally as incidents are resolved: IcM tickets, diagnostic log snippets, Kusto query results, and the resolution notes left by engineers who fixed the problem. The system processes this material through five stages. *Collection* gathers the relevant signals for a given monitor or service using monitor IDs or custom Kusto queries. *Filtering* applies machine learning to remove noise—duplicate signals, false positives, entries from unrelated incidents—and surface the attributes most predictive of resolution path. *Core Incident Selection* picks a small, representative subset of incidents that together cover the distinct failure modes associated with a given service, avoiding redundancy while ensuring coverage. *Data Distillation* reads the resolution paths of those core incidents and extracts the diagnostic checks and mitigation steps that actually worked. *TSG Generation* assembles the distilled steps into a structured, action-oriented guide using a large language model, formatted so that both human engineers and automated triage agents can consume it directly.

That last point matters for the integrated pipeline. Because TSGen's output is machine-readable and consistently structured, TRIANGLE agents can be trained on the generated corpus without any additional formatting work. In pilot deployments at Microsoft, dozens of AI-generated TSGs passed review and were published for production on-call use—a high bar, since engineers are under no obligation to accept machine-generated documentation and tend to be skeptical of it.

IV. TRIANGLE: Multi-Agent Negotiation for Incident Triage

Getting an incident to the wrong team is expensive. The wrong team spends time reading a ticket that isn't theirs, figures out it belongs elsewhere, and re-routes it. Time-to-engage—the interval between an alert firing and the right engineer starting to work—is the metric that takes the hit, and in a serious outage that delay is measured in user impact.

TRIANGLE approaches triage as a multi-agent reasoning problem [2]. Each engineering team has an agent that encodes that team's domain knowledge through two sources: its historical incident corpus and the TSGs generated by TSGen. When a new incident arrives, the system does not try to classify it with a single model trained on a static snapshot of team boundaries. Instead, it asks each agent whether the incident belongs to its team, collects the reasoning alongside the vote, and runs a negotiation phase in which agents revise their positions in light of each other's arguments until consensus emerges.

Three problems make naive classification fail here. *Semantic heterogeneity* means that the same root cause—say, a connection pool exhaustion in a shared database—can look completely different in text depending on which surface it manifests on and who filed the ticket. *Decentralized knowledge* means that the correct owner of an incident is often not determinable from any single team's perspective alone; a network misconfiguration that causes latency on a storage service is technically a networking team issue even if the storage team sees it first. *Dynamic responsibilities* means that team ownership changes as organizations restructure, and any static classifier trained three months ago may already be wrong. The negotiation mechanism handles all three: by letting agents reason over shared artifacts and revise their conclusions, the system adapts to ambiguity and organizational change without retraining.

In production, TRIANGLE achieves up to 97% triage accuracy and has reduced time-to-engage by up to 91% across a platform serving tens of millions of users.

V. KG-Graphlet: Graph-Structured Root Cause Analysis

Once the right team is engaged, they still face the hardest part: figuring out why the incident happened. The answer is almost always somewhere in the organization's accumulated history of post-mortem documents—Problem Review Board (PRB) records that describe past incidents, their root causes, and how they were resolved. The problem is that this knowledge is buried in natural language, spread across thousands of documents, and essentially unsearchable with keyword queries for anything beyond the most common failure modes [3].

The approach taken here builds a *Causal Knowledge Graph* (CKG) over the full PRB corpus by extracting entities—symptoms, root causes, resolutions, and services involved—and connecting them with typed edges that encode causal and temporal relationships. This turns an unstructured document archive into a queryable graph where semantic clustering over node embeddings reveals structure: in practice, 60 distinct symptom clusters emerge, each grouping incidents that share underlying failure patterns even when the surface descriptions are very different.

IcM Alert Fired

Stage 1 — TRIANGLE (Triage)

semantic distillation → agent negotiation

⇒ route to responsible team

Stage 2 — KG-Graphlet (Root Cause)

CKG query → graphlet matching

⇒ root cause hypothesis surfaced

Stage 3 — TSGen (Resolution)

corpus lookup or on-demand TSG synthesis

⇒ structured guide retrieved or generated

Engineer Confirms and Executes Mitigation

Resolved incident → new PRB record →

feeds back into CKG, TSGen, and TRIANGLE

Fig. 1. The GraphAIOps end-to-end pipeline. Each resolved incident produces artifacts that improve all three stages in the next cycle.

The most technically interesting contribution in this line of work is *graphlet inference*. Rather than finding nodes similar to the current incident, the system searches for recurring small subgraph patterns—graphlets— that capture how failures propagate through the service dependency graph. A graphlet might encode the pattern: "service A's error rate spikes, database B's connection count hits its ceiling 90 seconds later, and downstream service C begins returning 503s." This pattern can match even if A, B, and C have different names in the new incident, as long as the structural relationship is the same. Graphlet matching generalizes from experience in a way that node-level similarity cannot, because it captures the *mechanism* of failure rather than its surface appearance.

A GPT-3.5 model fine-tuned on prompts constructed from CKG subgraphs improves root cause generation quality by 45.5% over zero-shot prompting, and mitigation suggestion quality by 131.3%. In a live production evaluation, over 70% of on-call engineers rated the system's recommendations 3 out of 5 or higher— meaningful, given that experienced engineers are deeply skeptical of automated RCA and set a high bar for what counts as useful.

VI. GraphAIOps: Integration and Feedback

Figure 1 shows the full pipeline. When an alert fires, TRIANGLE routes it to the right team in seconds rather than hours. The CKG is immediately queried using the incident's entity signatures, and the graphlet matcher surfaces a root cause hypothesis along with the historical incidents that support it. TSGen's corpus is searched for the most relevant guide; if a close match exists, it is retrieved; if not, TSGen synthesizes one from similar resolved incidents on demand. The engineer reviews the hypothesis and the guide, then executes or adjusts the suggested mitigation.

What makes this arrangement more than a pipeline is the feedback loop that closes after each resolution. A resolved incident produces a new PRB record, which the CKG ingests to enrich its graphlet library. The resolution steps feed back into TSGen as new training material, keeping the guide corpus fresh without any manual effort. The updated TSGs flow back into the knowledge base that TRIANGLE agents draw on for the next triage cycle. The system does not require separate maintenance teams for each component—it improves through use.

Table I reports the measured impact of each component from independent production evaluations. The numbers do not compound trivially, but even independently they represent substantial improvements to incident response.

TABLE I
Measured Production Impact of Each Component

System	Metric	Result
TSGen	Mitigation time reduction	-40%
TRIANGLE	Triage accuracy	up to 97%
TRIANGLE	Time-to-engage reduction	-91%
KG-Graphlet	Root cause quality gain	$+45.5\%$
KG-Graphlet	Mitigation suggestion gain	$+131.3\%$
KG-Graphlet	Engineer satisfaction ($\geq$3/5)	$> 70\%$

VII. Discussion

The shared architectural choice across all three systems—representing incident knowledge as a graph rather than a flat corpus—is not arbitrary. Incident data is fundamentally relational. An alert does not have meaning in isolation; it has meaning in the context of which services depend on which other services, which teams own which components, and which failure patterns have occurred before under similar conditions. Flat retrieval treats all of that context as metadata to be discarded or encoded as feature vectors. Graph representations preserve it in a form that supports structured reasoning.

Several open problems remain. *Cold-start* is genuine: a new service with no incident history has no PRB records in the CKG, no resolved incidents for TSGen to learn from, and no base for TRIANGLE's agent. Transfer learning across services and organizations could address this, but the right inductive bias for cross-service failure pattern transfer is an open research question. *Agent drift* in TRIANGLE is also real: as team responsibilities change, an agent's domain knowledge becomes stale, and retraining requires curating a fresh corpus of accepted and rejected incidents—a non-trivial operational burden. Finally, both the CKG and TSGen are evaluated on proprietary Microsoft corpora, which makes independent benchmarking impossible for the research community. Publishing an anonymized incident dataset with synthetic entity labels would be a significant contribution to the field, independent of any system paper.

Looking ahead, the most impactful direction is federated incident learning: sharing graphlet patterns across organizations without exposing raw telemetry. A smaller cloud operator

running the same database software as a hyperscaler could benefit from failure propagation patterns discovered at scale, even if the specific service names differ. The graph representation used by the CKG is well-suited to this kind of abstracted knowledge transfer in a way that raw log sharing is not.

VIII. Conclusion

GraphAIOps brings together three production AI systems that each solve a distinct but interconnected problem in cloud incident management. TSGen ensures that troubleshooting guides exist, are current, and are structured for both humans and machines. TRIANGLE ensures that the right team engages with an incident immediately, without the round-trips that mis-triage causes. KG-Graphlet inference gives that team a concrete root cause hypothesis grounded in the organization's full history of past incidents, not just the engineer's personal experience.

What this paper argues is that these three systems are stronger together than apart—not just because their outputs compose cleanly, but because each system's artifacts improve the others in a cycle. Resolved incidents become better TSGs, better CKG entries, and better agent training data. The pipeline learns from every incident it handles, and over time it should close the gap between the median engineer's response and the best engineer's response. That is a genuinely ambitious goal for operational AI, and the production results from Microsoft suggest it is within reach.

References

[1] C. Bansal et al., "TSGen: Automated Troubleshooting Guide Generation from Cloud Incidents," in *Proc. FSE*, Montreal, Canada, Jul. 2026.

[2] Z. Yu, M. Ma, X. Feng, R. Ding, C. Zhang, Z. Li, M. Chintalapati, X. Zhang, R. Wang, C. Bansal, S. Rajmohan, Q. Lin, S. Zhang, C. Pei, and D. Pei, "TRIANGLE: Automated Incident Triaging for Cloud Reliability Data," in *Proc. ASE*, 2025.

[3] C. Bansal et al., "Mining Root Cause Knowledge from Cloud Service Incident Investigations for AIOps," *arXiv:2204.11598*, 2022.

[4] M. Shetty, C. Bansal, S. P. Upadhyayula, A. Radhakrishna, and A. Gupta, "AutoTSG: Machine Learning and Rule Based Framework for Automated Troubleshooting," in *Proc. ESEC/FSE*, 2022.

[5] X. Chen et al., "RCACopilot: On-Call Incident Root Cause Analysis with LLMs," in *Proc. EuroSys*, 2024.

[6] M. Ma et al., "DeCaf: Diagnosing and Triaging Performance Issues in Large-Scale Cloud Services," in *Proc. ICSE*, 2020.

[7] Z. Guo et al., "Nissist: An Incident Mitigation Copilot Based on Troubleshooting Guides," *arXiv:2402.17531*, 2024.

[8] X. Liu et al., "StepFly: An Agentic Framework for Troubleshooting Guide Automation," *arXiv:2510.10074*, 2025.

[9] "LLexus: LLM Agents for Executable TSG Plans," *Tech. Report, Microsoft Research*, 2024.